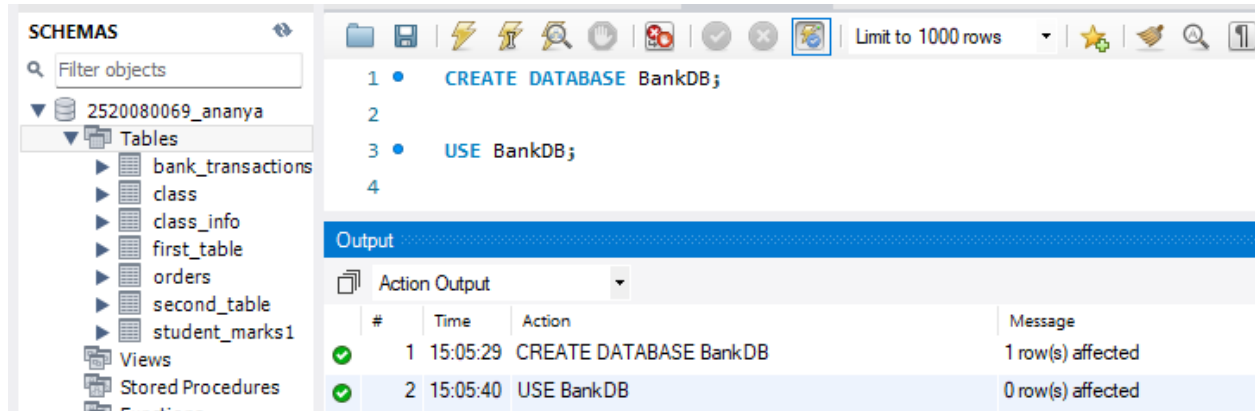


SKILL WEEK - 4 : VIEWS

1.Create Database :

CREATE DATABASE BankDB;

USE BankDB;



2. Create Customer table :

CREATE TABLE Customer (

Customer_ID INT PRIMARY KEY,

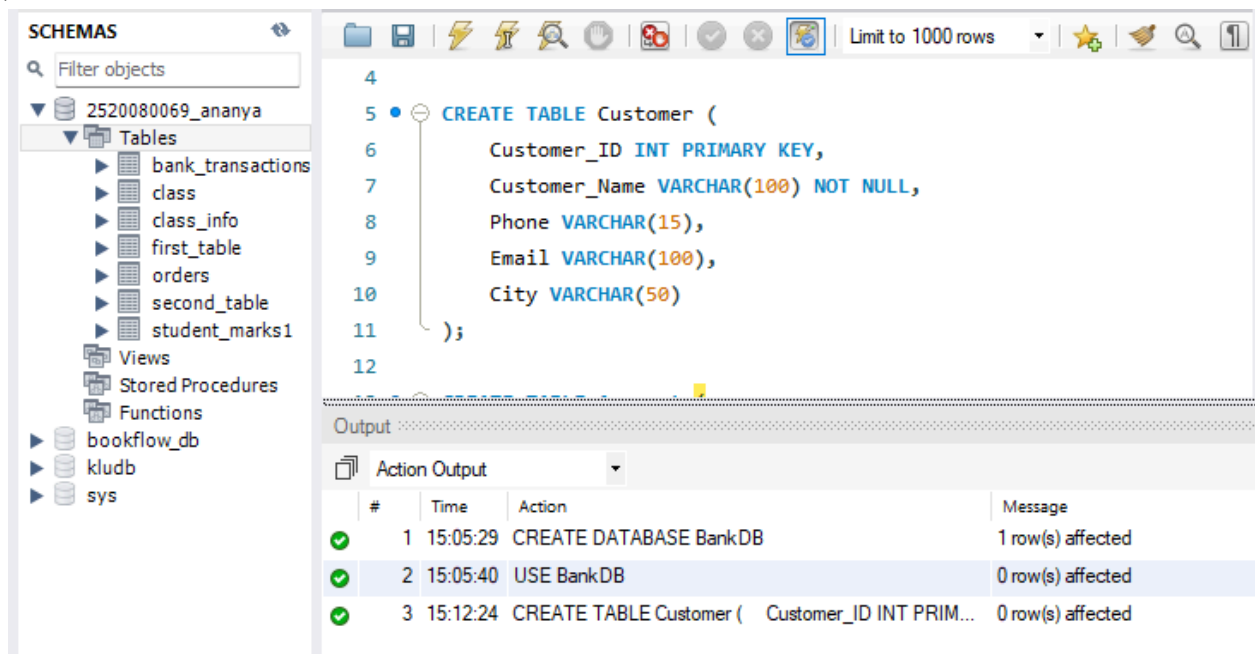
Customer_Name VARCHAR(100) NOT NULL,

Phone VARCHAR(15),

Email VARCHAR(100),

City VARCHAR(50)

);



3. Create Account table :

```
CREATE TABLE Account (  
    Account_No INT PRIMARY KEY,  
    Customer_ID INT,  
    Account_Type VARCHAR(20),  
    Balance DECIMAL(12,2),  
    Branch VARCHAR(50),  
  
    FOREIGN KEY (Customer_ID)  
    REFERENCES Customer(Customer_ID)  
);
```

The screenshot shows a database management tool interface. On the left, a 'SCHEMAS' panel lists various databases and tables. The main editor area displays the SQL code for creating the 'Account' table, which is highlighted in blue. The code is as follows:

```
12  
13 CREATE TABLE Account (  
14     Account_No INT PRIMARY KEY,  
15     Customer_ID INT,  
16     Account_Type VARCHAR(20),  
17     Balance DECIMAL(12,2),  
18     Branch VARCHAR(50),  
19  
20     FOREIGN KEY (Customer_ID)  
21     REFERENCES Customer(Customer_ID)  
22 );  
23
```

Below the editor, an 'Output' panel shows the execution results. It includes a table with columns for '#', 'Time', 'Action', and 'Message'.

#	Time	Action	Message
✓ 1	15:05:29	CREATE DATABASE BankDB	1 row(s) affected
✓ 2	15:05:40	USE BankDB	0 row(s) affected
✓ 3	15:12:24	CREATE TABLE Customer (Customer_ID INT PRIM...	0 row(s) affected
✓ 4	15:14:54	CREATE TABLE Account (Account_No INT PRIMA...	0 row(s) affected

4. Create Bank transaction table :

```
CREATE TABLE Bank_Transaction (  
    Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Account_No INT,  
    Transaction_Type VARCHAR(20),  
    Amount DECIMAL(12,2),  
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```

FOREIGN KEY (Account_No)
REFERENCES Account(Account_No)
);

```

SCHEMAS

Filter objects

2520080069_ananya

- Tables
 - bank_transactions
 - class
 - class_info
 - first_table
 - orders
 - second_table
 - student_marks1
- Views
- Stored Procedures
- Functions
- bookflow_db
- kludb
- sys

```

22 );
23
24 CREATE TABLE Bank_Transaction (
25     Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
26     Account_No INT,
27     Transaction_Type VARCHAR(20),
28     Amount DECIMAL(12,2),
29     Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
30
31     FOREIGN KEY (Account_No)
32     REFERENCES Account(Account_No)
33 );

```

Output

Action Output

#	Time	Action	Message
1	15:05:29	CREATE DATABASE BankDB	1 row(s) affected
2	15:05:40	USE BankDB	0 row(s) affected
3	15:12:24	CREATE TABLE Customer (Customer_ID INT PRIM...	0 row(s) affected
4	15:14:54	CREATE TABLE Account (Account_No INT PRIMA...	0 row(s) affected
5	15:16:00	CREATE TABLE Bank_Transaction (Transaction_ID...	0 row(s) affected

Administration Schemas

Information:.....

Schema:
2520080069_ananya

5. Create Laon table :

```

CREATE TABLE Loan (
    Loan_ID INT PRIMARY KEY,
    Customer_ID INT,
    Loan_Type VARCHAR(30),
    Loan_Amount DECIMAL(12,2),
    Interest_Rate DECIMAL(5,2),

    FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID)
);

```

SCHEMAS

2520080069_ananya

Tables

bank_transactions

class

class_info

first_table

orders

second_table

student_marks1

Views

Stored Procedures

Functions

bookflow_db

kludb

sys

Administration

Schemas

Information

Schema:

2520080069_ananya

Limit to 1000 rows

34

35

36

37

38

39

40

41

42

43

44

45

```

CREATE TABLE Loan (
    Loan_ID INT PRIMARY KEY,
    Customer_ID INT,
    Loan_Type VARCHAR(30),
    Loan_Amount DECIMAL(12,2),
    Interest_Rate DECIMAL(5,2),

    FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID)
);

```

Output

Action Output

#	Time	Action	Message
✓ 1	15:05:29	CREATE DATABASE BankDB	1 row(s) affected
✓ 2	15:05:40	USE BankDB	0 row(s) affected
✓ 3	15:12:24	CREATE TABLE Customer (Customer_ID INT PRIM...	0 row(s) affected
✓ 4	15:14:54	CREATE TABLE Account (Account_No INT PRIMA...	0 row(s) affected
✓ 5	15:16:00	CREATE TABLE Bank_Transaction (Transaction_ID...	0 row(s) affected
✓ 6	15:17:11	CREATE TABLE Loan (Loan_ID INT PRIMARY KE...	0 row(s) affected

6. Insert Customer data :

```
INSERT INTO Customer
(Customer_ID, Customer_Name, Phone, Email, City)
VALUES
(101, 'Ravi Kumar', '9876543210',
'ravi@gmail.com', 'Hyderabad'),

(102, 'Priya Sharma', '9876543211',
'priya@gmail.com', 'Vijayawada'),

(103, 'Arjun Reddy', '9876543212',
'arjun@gmail.com', 'Bangalore'),

(104, 'Sneha Rao', '9876543213',
'sneha@gmail.com', 'Chennai'),

(105, 'Kiran Kumar', '9876543214',
'kiran@gmail.com', 'Hyderabad'),

(106, 'Anil Kumar', '9876543215',
'anil@gmail.com', 'Delhi'),

(107, 'Meena Reddy', '9876543216',
'meena@gmail.com', 'Mumbai'),

(108, 'Rahul Sharma', '9876543217',
'rahul@gmail.com', 'Pune'),

(109, 'Lakshmi Devi', '9876543218',
'lakshmi@gmail.com', 'Hyderabad'),

(110, 'Suresh Babu', '9876543219',
'suresh@gmail.com', 'Vijayawada');
```

SCHEMAS

Filter objects

2520080069_ananya

Tables

bank_transactions

class

class_info

first_table

orders

second_table

student_marks1

Views

Stored Procedures

Functions

bookflow_db

kludb

sys

Administration

Schemas

Information

Schema: 2520080069_ananya

Limit to 1000 rows

Automatic disabled. Use manually g current care toggle au

```
46 INSERT INTO Customer
47 (Customer_ID, Customer_Name, Phone, Email, City)
48 VALUES
49 (101, 'Ravi Kumar', '9876543210',
50 'ravi@gmail.com', 'Hyderabad'),
51
52 (102, 'Priya Sharma', '9876543211',
53 'priya@gmail.com', 'Vijayawada'),
54
55 (103, 'Arjun Reddy', '9876543212',
56 'arjun@gmail.com', 'Bangalore'),
57
58 (104, 'Sneha Rao', '9876543213',
59 'sneha@gmail.com', 'Chennai'),
60
61 (105, 'Kiran Kumar', '9876543214',
62 'kiran@gmail.com', 'Hyderabad'),
63
64 (106, 'Anil Kumar', '9876543215',
65 'anil@gmail.com', 'Delhi'),
66
67 (107, 'Meena Reddy', '9876543216',
68 'meena@gmail.com', 'Mumbai'),
69
70 (108, 'Rahul Sharma', '9876543217',
71 'rahul@gmail.com', 'Pune'),
72
73 (109, 'Lakshmi Devi', '9876543218',
74 'lakshmi@gmail.com', 'Hyderabad'),
75
76 (110, 'Suresh Babu', '9876543219',
77 'suresh@gmail.com', 'Vijayawada');
```

Output

Action Output

#	Time	Action	Message
✓ 2	15:05:40	USE BankDB	0 row(s) affected
✓ 3	15:12:24	CREATE TABLE Customer (Customer_ID INT PRI...	0 row(s) affected
✓ 4	15:14:54	CREATE TABLE Account (Account_No INT PRIM...	0 row(s) affected
✓ 5	15:16:00	CREATE TABLE Bank_Transaction (Transaction_I...	0 row(s) affected
✓ 6	15:17:11	CREATE TABLE Loan (Loan_ID INT PRIMARY K...	0 row(s) affected
✗ 7	15:19:13	INSERT INTO Customer (Customer_ID, Customer_Na...	Error Code: 1064. You have an error in your SQL synta...
✓ 8	15:21:28	INSERT INTO Customer (Customer_ID, Customer_Na...	10 row(s) affected Records: 10 Duplicates: 0 Wamin...

Object Info

Session

7. Insert Account data :

INSERT INTO Account

(Account_No, Customer_ID, Account_Type, Balance, Branch)

VALUES

(10001, 101, 'Savings', 50000, 'Hyderabad'),

(10002, 102, 'Savings', 75000, 'Vijayawada'),

(10003, 103, 'Current', 120000, 'Bangalore'),

(10004, 104, 'Savings', 45000, 'Chennai'),

(10005, 105, 'Current', 90000, 'Hyderabad'),

(10006, 106, 'Savings', 65000, 'Delhi'),

(10007, 107, 'Current', 150000, 'Mumbai'),

(10008, 108, 'Savings', 35000, 'Pune'),

(10009, 109, 'Savings', 85000, 'Hyderabad'),

(10010, 110, 'Current', 110000, 'Vijayawada');

SCHEMAS

Filter objects

2520080069_ananya

Tables

bank_transactions

class

class_info

first_table

orders

second_table

student_marks1

Views

Stored Procedures

Functions

bookflow_db

kludb

sys

Administration

Schemas

Information

Schema:

2520080069_ananya

Limit to 1000 rows

76 (110, 'Suresh Babu', '9876543219',

77 'suresh@gmail.com', 'Vijayawada');

78

79

80 INSERT INTO Account

81 (Account_No, Customer_ID, Account_Type, Balance, Branch)

82 VALUES

83 (10001, 101, 'Savings', 50000, 'Hyderabad'),

84

85 (10002, 102, 'Savings', 75000, 'Vijayawada'),

86

87 (10003, 103, 'Current', 120000, 'Bangalore'),

88

89 (10004, 104, 'Savings', 45000, 'Chennai'),

90

91 (10005, 105, 'Current', 90000, 'Hyderabad'),

92

93 (10006, 106, 'Savings', 65000, 'Delhi'),

94

95 (10007, 107, 'Current', 150000, 'Mumbai'),

96

97 (10008, 108, 'Savings', 35000, 'Pune'),

98

99 (10009, 109, 'Savings', 85000, 'Hyderabad'),

100

101 (10010, 110, 'Current', 110000, 'Vijayawada');

102

103 INSERT INTO Bank_Transaction

104 (Account_No, Transaction_Type, Amount)

105 VALUES

106 (10001, 'DEPOSIT', 10000),

107

Automatic
disabled. Us
manually g
current care
toggle au

Context Help

Snip

Output

Action Output

#	Time	Action	Message
✓ 3	15:12:24	CREATE TABLE Customer (Customer_ID INT PRI...	0 row(s) affected
✓ 4	15:14:54	CREATE TABLE Account (Account_No INT PRIM...	0 row(s) affected
✓ 5	15:16:00	CREATE TABLE Bank_Transaction (Transaction_I...	0 row(s) affected
✓ 6	15:17:11	CREATE TABLE Loan (Loan_ID INT PRIMARY K...	0 row(s) affected
✗ 7	15:19:13	INSERT INTO Customer (Customer_ID, Customer_Na...	Error Code: 1064. You have an error in your SQL synta...
✓ 8	15:21:28	INSERT INTO Customer (Customer_ID, Customer_Na...	10 row(s) affected Records: 10 Duplicates: 0 Wamin...
✓ 9	15:23:04	INSERT INTO Account (Account_No, Customer_ID, ...	10 row(s) affected Records: 10 Duplicates: 0 Wamin...

Object Info

Session

8. Insert Bank transaction data :

```
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 10000),

(10001, 'WITHDRAW', 5000),

(10002, 'DEPOSIT', 15000),

(10003, 'WITHDRAW', 20000),

(10004, 'DEPOSIT', 5000),

(10005, 'WITHDRAW', 10000),

(10006, 'DEPOSIT', 12000),

(10007, 'DEPOSIT', 25000),

(10008, 'WITHDRAW', 5000),

(10009, 'DEPOSIT', 20000),

(10010, 'WITHDRAW', 15000);
```

SCHEMAS

Filter objects

- 2520080069_ananya
 - Tables
 - bank_transactions
 - class
 - class_info
 - first_table
 - orders
 - second_table
 - student_marks1
 - Views
 - Stored Procedures
 - Functions
- bookflow_db
- kludb
- sys

Administration Schemas

Information:.....

Schema:
2520080069_ananya

Object Info Session

```
100
101 (10010, 110, 'Current', 110000, 'Vijayawada');
102
103 INSERT INTO Bank_Transaction
104 (Account_No, Transaction_Type, Amount)
105 VALUES
106 (10001, 'DEPOSIT', 10000),
107
108 (10001, 'WITHDRAW', 5000),
109
110 (10002, 'DEPOSIT', 15000),
111
112 (10003, 'WITHDRAW', 20000),
113
114 (10004, 'DEPOSIT', 5000),
115
116 (10005, 'WITHDRAW', 10000),
117
118 (10006, 'DEPOSIT', 12000),
119
120 (10007, 'DEPOSIT', 25000),
121
122 (10008, 'WITHDRAW', 5000),
123
124 (10009, 'DEPOSIT', 20000),
125
126 (10010, 'WITHDRAW', 15000);
127
128
129 INSERT INTO Loan
130 (Loan_ID, Customer_ID, Loan_Type,
131 Loan_Amount, Interest_Rate)
```

Automatic
disabled. U
manually
current ca
toggle a

Context Help Sn

Output

Action Output

#	Time	Action	Message
✓ 4	15:14:54	CREATE TABLE Account (Account_No INT PRIM...	0 row(s) affected
✓ 5	15:16:00	CREATE TABLE Bank_Transaction (Transaction_I...	0 row(s) affected
✓ 6	15:17:11	CREATE TABLE Loan (Loan_ID INT PRIMARY K...	0 row(s) affected
✗ 7	15:19:13	INSERT INTO Customer (Customer_ID, Customer_Na...	Error Code: 1064. You have an error in your SQL synta...
✓ 8	15:21:28	INSERT INTO Customer (Customer_ID, Customer_Na...	10 row(s) affected Records: 10 Duplicates: 0 Wamin...
✓ 9	15:23:04	INSERT INTO Account (Account_No, Customer_ID, ...	10 row(s) affected Records: 10 Duplicates: 0 Wamin...
✓ 10	15:24:37	INSERT INTO Bank_Transaction (Account_No, Tran...	11 row(s) affected Records: 11 Duplicates: 0 Wamin...

9. Insert loan data :

```
INSERT INTO Loan
(Loan_ID, Customer_ID, Loan_Type,
 Loan_Amount, Interest_Rate)
VALUES
(501, 101, 'Home Loan', 5000000, 7.5),

(502, 102, 'Education Loan', 1000000, 6.5),

(503, 103, 'Car Loan', 800000, 8.2),

(504, 104, 'Personal Loan', 500000, 10.5),

(505, 105, 'Home Loan', 4000000, 7.2),

(506, 106, 'Car Loan', 900000, 8.5),

(507, 107, 'Business Loan', 3000000, 9.0),

(508, 109, 'Personal Loan', 600000, 10.0);
```

SCHEMAS

Filter objects

2520080069_ananya

Tables

bank_transactions

class

class_info

first_table

orders

second_table

student_marks1

Views

Stored Procedures

Functions

bookflow_db

kludb

sys

Administration Schemas

Information

Schema:
2520080069_ananya

Query 1

Limit to 1000 rows

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

INSERT INTO Loan

(Loan_ID, Customer_ID, Loan_Type,

Loan_Amount, Interest_Rate)

VALUES

(501, 101, 'Home Loan', 5000000, 7.5),

(502, 102, 'Education Loan', 1000000, 6.5),

(503, 103, 'Car Loan', 800000, 8.2),

(504, 104, 'Personal Loan', 500000, 10.5),

(505, 105, 'Home Loan', 4000000, 7.2),

(506, 106, 'Car Loan', 900000, 8.5),

(507, 107, 'Business Loan', 3000000, 9.0),

(508, 109, 'Personal Loan', 600000, 10.0);

Automatic color
disabled. Use t
manually get
current caret p
toggle auto

Context Help Snippets

Output

Action Output

#	Time	Action	Message
1	15:05:29	CREATE DATABASE BankDB	1 row(s) affected
2	15:05:40	USE BankDB	0 row(s) affected
3	15:12:24	CREATE TABLE Customer (Customer_ID INT PRIM...	0 row(s) affected
4	15:14:54	CREATE TABLE Account (Account_No INT PRIMA...	0 row(s) affected
5	15:16:00	CREATE TABLE Bank_Transaction (Transaction_ID...	0 row(s) affected
6	15:17:11	CREATE TABLE Loan (Loan_ID INT PRIMARY KE...	0 row(s) affected
7	15:19:13	INSERT INTO Customer (Customer_ID, Customer_Nam...	Error Code: 1064. You have an error in your SQL syntax:...
8	15:21:28	INSERT INTO Customer (Customer_ID, Customer_Nam...	10 row(s) affected Records: 10 Duplicates: 0 Warnings...
9	15:23:04	INSERT INTO Account (Account_No, Customer_ID, Ac...	10 row(s) affected Records: 10 Duplicates: 0 Warnings...
10	15:24:37	INSERT INTO Bank_Transaction (Account_No, Transa...	11 row(s) affected Records: 11 Duplicates: 0 Warnings...
11	15:25:55	INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Typ...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0

10. Display the tables created :

SELECT * FROM Customer;

Administration Schemas

Information:.....

Schema:
2520080069_ananya

165
166

Result Grid

Filter Rows: Edit: Export/Import:

	Customer_ID	Customer_Name	Phone	Email	City
▶	101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
	102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
	103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
	104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
	105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
	106	Anil Kumar	9876543215	anil@gmail.com	Delhi
	107	Meena Reddy	9876543216	meena@gmail.com	Mumbai
	108	Rahul Sharma	9876543217	rahul@gmail.com	Pune
	109	Lakshmi Devi	9876543218	lakshmi@gmail.com	Hyderabad
	110	Suresh Babu	9876543219	suresh@gmail.com	Vijayawada
*	NULL	NULL	NULL	NULL	NULL

Customer 1 x Account 2 Bank_Transaction 3 Loan 4 Apply Revert

SELECT * FROM Account;

Information:.....

Schema:
2520080069_ananya

Result Grid

Filter Rows: Edit: Export/Import:

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	50000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
	10003	103	Current	120000.00	Bangalore
	10004	104	Savings	45000.00	Chennai
	10005	105	Current	90000.00	Hyderabad
	10006	106	Savings	65000.00	Delhi
	10007	107	Current	150000.00	Mumbai
	10008	108	Savings	35000.00	Pune
	10009	109	Savings	85000.00	Hyderabad
	10010	110	Current	110000.00	Vijayawada
*	NULL	NULL	NULL	NULL	NULL

Customer 1 Account 2 x Bank_Transaction 3 Loan 4 Apply Revert

SELECT * FROM Bank_Transaction;

Schema:
2520080069_ananya

Result Grid

Filter Rows:

Edit:

Export/Import:

	Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
▶	1	10001	DEPOSIT	10000.00	2026-08-12 15:24:37
	2	10001	WITHDRAW	5000.00	2026-08-12 15:24:37
	3	10002	DEPOSIT	15000.00	2026-08-12 15:24:37
	4	10003	WITHDRAW	20000.00	2026-08-12 15:24:37
	5	10004	DEPOSIT	5000.00	2026-08-12 15:24:37
	6	10005	WITHDRAW	10000.00	2026-08-12 15:24:37
	7	10006	DEPOSIT	12000.00	2026-08-12 15:24:37
	8	10007	DEPOSIT	25000.00	2026-08-12 15:24:37
	9	10008	WITHDRAW	5000.00	2026-08-12 15:24:37
	10	10009	DEPOSIT	20000.00	2026-08-12 15:24:37
	11	10010	WITHDRAW	15000.00	2026-08-12 15:24:37
✱	NULL	NULL	NULL	NULL	NULL

Customer 1

Account 2

Bank_Transaction 3

✕

Loan 4

Apply

SELECT * FROM Loan;

Schema:
2520080069_ananya

Result Grid

Filter Rows:

Edit:

Export/Import:

	Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
▶	501	101	Home Loan	5000000.00	7.50
	502	102	Education Loan	1000000.00	6.50
	503	103	Car Loan	800000.00	8.20
	504	104	Personal Loan	500000.00	10.50
	505	105	Home Loan	4000000.00	7.20
	506	106	Car Loan	900000.00	8.50
	507	107	Business Loan	3000000.00	9.00
	508	109	Personal Loan	600000.00	10.00
*	NULL	NULL	NULL	NULL	NULL

Customer 1

Account 2

Bank_Transaction 3

Loan 4 ×

Apply

Revert

Result Grid

Form Editor

Field Types

Query Stats

11. Create and Display view for all customers :

```
CREATE VIEW Customer_View AS
```

```
SELECT *
```

```
FROM Customer;
```

```
SELECT * FROM Customer_View;
```

The screenshot shows a database management interface. On the left, there's a sidebar with 'Administration' and 'Schemas' tabs. Below 'Schemas', it says 'Schema: 2520080069_ananya'. The main area displays SQL code in a text editor with line numbers 156 to 161. The code is: 156, 157 • CREATE VIEW Customer_View AS, 158 SELECT *, 159 FROM Customer;, 160, 161 • SELECT * FROM Customer_View;. Below the code editor is a 'Result Grid' tab. It shows a table with 6 columns: Customer_ID, Customer_Name, Phone, Email, and City. There are 10 rows of data. At the bottom of the result grid, it says 'Customer_View 5 x' and 'Read Only'.

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
106	Anil Kumar	9876543215	anil@gmail.com	Delhi
107	Meena Reddy	9876543216	meena@gmail.com	Mumbai
108	Rahul Sharma	9876543217	rahul@gmail.com	Pune
109	Lakshmi Devi	9876543218	lakshmi@gmail.com	Hyderabad
110	Suresh Babu	9876543219	suresh@gmail.com	Vijayawada

12. Create and Display view for customers basic information :

```
CREATE VIEW Customer_Basic_View AS
```

```
SELECT
```

```
    Customer_ID,
```

```
    Customer_Name,
```

```
    City
```

```
FROM Customer;
```

```
SELECT * FROM Customer_Basic_View;
```

Views
Stored Procedures
Functions
bookflow_db
kludb
sys

Administration Schemas
Information

Schema:
2520080069_ananya

```

162
163 • CREATE VIEW Customer_Basic_View AS
164 SELECT
165     Customer_ID,
166     Customer_Name,
167     City
168 FROM Customer;
169
170
171 SELECT * FROM Customer_Basic_View;
172

```

Result Grid | Filter Rows: | Export: | W

	Customer_ID	Customer_Name	City
▶	101	Ravi Kumar	Hyderabad
	102	Priya Sharma	Vijayawada
	103	Arjun Reddy	Bangalore
	104	Sneha Rao	Chennai
	105	Kiran Kumar	Hyderabad
	106	Anil Kumar	Delhi
	107	Meena Reddy	Mumbai
	108	Rahul Sharma	Pune
	109	Lakshmi Devi	Hyderabad
	110	Suresh Babu	Vijayawada

13. Create and Display view for account information :

CREATE VIEW Account_View AS

SELECT

Account_No,

Account_Type,

Balance,

Branch

FROM Account;

SELECT * FROM Account_View;

The screenshot shows a database management interface. On the left, a tree view displays the database structure, including 'Views', 'Stored Procedures', and 'Functions'. Below this, the 'Schemas' tab is active, showing the schema '2520080069_ananya'. The main area displays SQL queries in a text editor. The queries are as follows:

```

163 • CREATE VIEW Customer_Basic_View AS
164 SELECT
165     Customer_ID,
166     Customer_Name,
167     City
168 FROM Customer;
169
170
171 SELECT * FROM Customer_Basic_View;
172
173 • CREATE VIEW Account_View AS

```

Below the queries, a 'Result Grid' is displayed, showing the results of the queries. The grid has columns for 'Account_No', 'Account_Type', 'Balance', and 'Branch'. The data is as follows:

Account_No	Account_Type	Balance	Branch
10001	Savings	50000.00	Hyderabad
10002	Savings	75000.00	Vijayawada
10003	Current	120000.00	Bangalore
10004	Savings	45000.00	Chennai
10005	Current	90000.00	Hyderabad
10006	Savings	65000.00	Delhi
10007	Current	150000.00	Mumbai
10008	Savings	35000.00	Pune
10009	Savings	85000.00	Hyderabad
10010	Current	110000.00	Vijayawada

14. Create and Display view for saving accounts :

```
CREATE VIEW Savings_Account_View AS
```

```
SELECT *
```

```
FROM Account
```

```
WHERE Account_Type = 'Savings';
```

```
SELECT * FROM Savings_Account_View;
```

Administration Schemas Information

Schema: 2520080069_ananya

```

182
183 • CREATE VIEW Savings_Account_View AS
184 SELECT *
185 FROM Account
186 WHERE Account_Type = 'Savings';
187
188 • SELECT * FROM Savings_Account_View;
189

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	50000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
	10004	104	Savings	45000.00	Chennai
	10006	106	Savings	65000.00	Delhi
	10008	108	Savings	35000.00	Pune
	10009	109	Savings	85000.00	Hyderabad

15. Create and Display view for current accounts :

CREATE VIEW Current_Account_View AS

SELECT *

FROM Account

WHERE Account_Type = 'Current';

SELECT * FROM Current_Account_View;

Administration Schemas Information

Schema: 2520080069_ananya

```

189
190 • CREATE VIEW Current_Account_View AS
191 SELECT *
192 FROM Account
193 WHERE Account_Type = 'Current';
194
195 • SELECT * FROM Current_Account_View;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10003	103	Current	120000.00	Bangalore
	10005	105	Current	90000.00	Hyderabad
	10007	107	Current	150000.00	Mumbai
	10010	110	Current	110000.00	Vijayawada

16. Create and Display view for high balance accounts :

```
CREATE VIEW High_Balance_View AS
```

```
SELECT
```

```
    Account_No,  
    Customer_ID,  
    Account_Type,  
    Balance
```

```
FROM Account
```

```
WHERE Balance > 100000;
```

```
SELECT * FROM High_Balance_View;
```

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Server Enterprise Explorer' tree shows the 'sys' database selected. The 'Administration' pane on the right shows the 'Schemas' tab. The 'Query Editor' window is open, showing the following SQL script:

```
196  
197 • CREATE VIEW High_Balance_View AS  
198 SELECT  
199     Account_No,  
200     Customer_ID,  
201     Account_Type,  
202     Balance  
203 FROM Account  
204 WHERE Balance > 100000;  
205  
206 • SELECT * FROM High_Balance_View;  
207
```

Below the script, the 'Result Grid' is displayed, showing the output of the query. The grid has five columns: 'Account_No', 'Customer_ID', 'Account_Type', and 'Balance'. The data is as follows:

	Account_No	Customer_ID	Account_Type	Balance
▶	10003	103	Current	120000.00
	10007	107	Current	150000.00
	10010	110	Current	110000.00

17. Create and Display view for hyderabad account :

```
CREATE VIEW Hyderabad_Account_View AS  
SELECT *  
FROM Account  
WHERE Branch = 'Hyderabad';
```

```
SELECT * FROM Hyderabad_Account_View;
```

The screenshot shows the SQL Developer interface. The 'Schemas' tab is selected, and the 'Administration' pane on the left shows 'No object selected'. The main editor displays the following SQL code:

```
208 • CREATE VIEW Hyderabad_Account_View AS  
209 SELECT *  
210 FROM Account  
211 WHERE Branch = 'Hyderabad';  
212  
213 • SELECT * FROM Hyderabad_Account_View;
```

Below the editor, the 'Result Grid' is displayed, showing the output of the executed query. The grid has the following columns: Account_No, Customer_ID, Account_Type, Balance, and Branch. The data is as follows:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10005	105	Current	90000.00	Hyderabad
10009	109	Savings	85000.00	Hyderabad

18. Create and Display view for low balance account :

```
CREATE VIEW Low_Balance_View AS  
SELECT  
    Account_No,  
    Customer_ID,  
    Balance  
FROM Account  
WHERE Balance < 50000;
```

```
SELECT * FROM Low_Balance_View;
```

The screenshot shows a database management interface. On the left, there are tabs for 'Administration' and 'Schemas', and a section labeled 'Information' with the text 'No object selected'. The main area displays SQL code with line numbers 214 through 224. The code defines a view 'Low_Balance_View' and then queries it. Below the code, there is a 'Result Grid' section with a table containing two rows of data.

```
214
215 • CREATE VIEW Low_Balance_View AS
216 SELECT
217     Account_No,
218     Customer_ID,
219     Balance
220 FROM Account
221 WHERE Balance < 50000;
222
223 • SELECT * FROM Low_Balance_View;
224
```

Account_No	Customer_ID	Balance
10004	104	45000.00
10008	108	35000.00

19. Customer + account view:

```
CREATE VIEW Customer_Account_View AS
SELECT
    C.Customer_ID,
    C.Customer_Name,
    C.City,
    A.Account_No,
    A.Account_Type,
    A.Balance,
    A.Branch
FROM Customer C
JOIN Account A
ON C.Customer_ID = A.Customer_ID;

SELECT * FROM Customer_Account_View;
```

225 • **CREATE VIEW Customer_Account_View AS**
 226 **SELECT**
 227 C.Customer_ID,
 228 C.Customer_Name,
 229 C.City,
 230 A.Account_No,
 231 A.Account_Type,
 232 A.Balance,
 233 A.Branch
 234 **FROM Customer C**
 235 **JOIN Account A**
 236 **ON C.Customer_ID = A.Customer_ID;**
 237
 238 • **SELECT * FROM Customer_Account_View;**

Admin:

Information:

No object selected

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch
▶	101	Ravi Kumar	Hyderabad	10001	Savings	50000.00	Hyderabad
	102	Priya Sharma	Vijayawada	10002	Savings	75000.00	Vijayawada
	103	Arjun Reddy	Bangalore	10003	Current	120000.00	Bangalore
	104	Sneha Rao	Chennai	10004	Savings	45000.00	Chennai
	105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad
	106	Anil Kumar	Delhi	10006	Savings	65000.00	Delhi
	107	Meena Reddy	Mumbai	10007	Current	150000.00	Mumbai
	108	Rahul Sharma	Pune	10008	Savings	35000.00	Pune
	109	Lakshmi Devi	Hyderabad	10009	Savings	85000.00	Hyderabad
	110	Suresh Babu	Vijayawada	10010	Current	110000.00	Vijayawada

Result Grid
Form Editor
Field Types

20. Hyderabad customer account view:

CREATE VIEW Hyderabad_Customer_Accounts AS

SELECT

 C.Customer_Name,

 C.City,

 A.Account_No,

 A.Account_Type,

 A.Balance

FROM Customer C

JOIN Account A

ON C.Customer_ID = A.Customer_ID

WHERE A.Branch = 'Hyderabad';

SELECT * FROM Hyderabad_Customer_Accounts;

```

240 • CREATE VIEW Hyderabad_Customer_Accounts AS
241 SELECT
242     C.Customer_Name,
243     C.City,
244     A.Account_No,
245     A.Account_Type,
246     A.Balance
247 FROM Customer C
248 JOIN Account A
249 ON C.Customer_ID = A.Customer_ID
250 WHERE A.Branch = 'Hyderabad';
251
252 • SELECT * FROM Hyderabad_Customer_Accounts;
253
254 • CREATE VIEW Customer_Loan_View AS
255 SELECT
256     C.Customer_ID,

```

Result Grid Filter Rows: Export: Wrap Cell Content:					
	Customer_Name	City	Account_No	Account_Type	Balance
▶	Ravi Kumar	Hyderabad	10001	Savings	50000.00
	Kiran Kumar	Hyderabad	10005	Current	90000.00
	Lakshmi Devi	Hyderabad	10009	Savings	85000.00

21. Customer loan view:

```
CREATE VIEW Customer_Loan_View AS
```

```
SELECT
```

```

    C.Customer_ID,
    C.Customer_Name,
    C.City,
    L.Loan_ID,
    L.Loan_Type,
    L.Loan_Amount,
    L.Interest_Rate

```

```
FROM Customer C
```

```
JOIN Loan L
```

```
ON C.Customer_ID = L.Customer_ID;
```

```
SELECT * FROM Customer_Loan_View;
```

The screenshot shows a database management tool interface. On the left, a tree view displays a database structure with folders for 'Fund', 'book', 'kludb', and 'sys'. The main area displays SQL code for creating and querying a view. The code is as follows:

```

239
240 • CREATE VIEW Hyderabad_Customer_Accounts AS
241 SELECT
242     C.Customer_Name,
243     C.City,
244     A.Account_No,
245     A.Account_Type,
246     A.Balance
247 FROM Customer C
248 JOIN Account A
249 ON C.Customer_ID = A.Customer_ID
250 WHERE A.Branch = 'Hyderabad';
251
252 • SELECT * FROM Hyderabad_Customer_Accounts;
  
```

Below the code, the results of the query are displayed in a table. The table has columns: Customer_Name, City, Account_No, Account_Type, and Balance. The results are as follows:

Customer_Name	City	Account_No	Account_Type	Balance
Ravi Kumar	Hyderabad	10001	Savings	50000.00
Kiran Kumar	Hyderabad	10005	Current	90000.00
Lakshmi Devi	Hyderabad	10009	Savings	85000.00

22. Complete Customer banking view:

CREATE VIEW Customer_Banking_View AS

SELECT

C.Customer_ID,
 C.Customer_Name,
 C.City,
 A.Account_No,
 A.Account_Type,
 A.Balance,
 A.Branch,
 L.Loan_Type,
 L.Loan_Amount

FROM Customer C

LEFT JOIN Account A

ON C.Customer_ID = A.Customer_ID

LEFT JOIN Loan L

ON C.Customer_ID = L.Customer_ID;

SELECT * FROM Customer_Banking_View;

Filter object

2520080

bankdb

Table

View

Store

Funcd

bookflow

kludb

sys

Adminis

Information

270

•

CREATE VIEW Customer_Banking_View AS

271

SELECT

272

C.Customer_ID,

273

C.Customer_Name,

274

C.City,

275

A.Account_No,

276

A.Account_Type,

277

A.Balance,

278

A.Branch,

279

L.Loan_Type,

280

L.Loan_Amount

281

FROM Customer C

282

LEFT JOIN Account A

283

ON C.Customer_ID = A.Customer_ID

284

LEFT JOIN Loan L

285

ON C.Customer_ID = L.Customer_ID;

286

287

•

SELECT * FROM Customer_Banking_View;

288

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch
▶	101	Ravi Kumar	Hyderabad	10001	Savings	50000.00	Hyderabad
	102	Priya Sharma	Vijayawada	10002	Savings	75000.00	Vijayawada
	103	Arjun Reddy	Bangalore	10003	Current	120000.00	Bangalore
	104	Sneha Rao	Chennai	10004	Savings	45000.00	Chennai
	105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad
	106	Anil Kumar	Delhi	10006	Savings	65000.00	Delhi
	107	Meena Reddy	Mumbai	10007	Current	150000.00	Mumbai
	108	Rahul Sharma	Pune	10008	Savings	35000.00	Pune
	109	Lakshmi Devi	Hyderabad	10009	Savings	85000.00	Hyderabad
	110	Suresh Babu	Vijayawada	10010	Current	110000.00	Vijayawada

Result Grid

Form Editor

Field Types

Query Stats

Customer Banking View 7

Read Only

No object selected

23. Total bank balance :

```
CREATE VIEW Total_Bank_Balance AS
SELECT
    SUM(Balance) AS Total_Balance
FROM Account;
```

```
SELECT * FROM Total_Bank_Balance;
```

The screenshot shows a SQL editor with the following code:

```
289 • CREATE VIEW Total_Bank_Balance AS
290 SELECT
291     SUM(Balance) AS Total_Balance
292 FROM Account;
293
294 • SELECT * FROM Total_Bank_Balance;
295
```

Below the editor, the 'Result Grid' tab is active, displaying the following data:

Total_Balance
825000.00

On the left side, there is a sidebar with 'Admini...' and 'Information' tabs. A message 'No object selected' is visible.

24. Average account balance:

```
CREATE VIEW Average_Account_Balance AS
SELECT
    AVG(Balance) AS Average_Balance
FROM Account;
```

```
SELECT * FROM Average_Account_Balance;
```

The screenshot shows a SQL editor with the following code:

```
295
296 • CREATE VIEW Average_Account_Balance AS
297 SELECT
298     AVG(Balance) AS Average_Balance
299 FROM Account;
300
301 • SELECT * FROM Average_Account_Balance;
302
```

Below the editor, the 'Result Grid' tab is active, displaying the following data:

Average_Balance
82500.000000

On the left side, there is a sidebar with 'Admini...' and 'Information' tabs. A message 'No object selected' is visible.

25. Maximum account balance :

```
CREATE VIEW Maximum_Balance_View AS  
SELECT  
    MAX(Balance) AS Maximum_Balance  
FROM Account;
```

```
SELECT * FROM Maximum_Balance_View;
```

The screenshot shows the SQL Developer interface. The SQL Editor contains the following queries:

```
303 • CREATE VIEW Maximum_Balance_View AS  
304 SELECT  
305     MAX(Balance) AS Maximum_Balance  
306 FROM Account;  
307  
308 • SELECT * FROM Maximum_Balance_View;  
309
```

The Results pane shows the output of the second query:

Maximum_Balance
150000.00

The interface also shows a toolbar with options like "Result Grid", "Filter Rows", "Export", and "Wrap".

26. Minimum account balance:

```
CREATE VIEW Minimum_Balance_View AS  
SELECT  
    MIN(Balance) AS Minimum_Balance  
FROM Account;  
SELECT * FROM Minimum_Balance_View;
```

The screenshot shows the SQL Developer interface. The SQL Editor contains the following queries:

```
309  
310 • CREATE VIEW Minimum_Balance_View AS  
311 SELECT  
312     MIN(Balance) AS Minimum_Balance  
313 FROM Account;  
314 • SELECT * FROM Minimum_Balance_View;  
315
```

The Results pane shows the output of the second query:

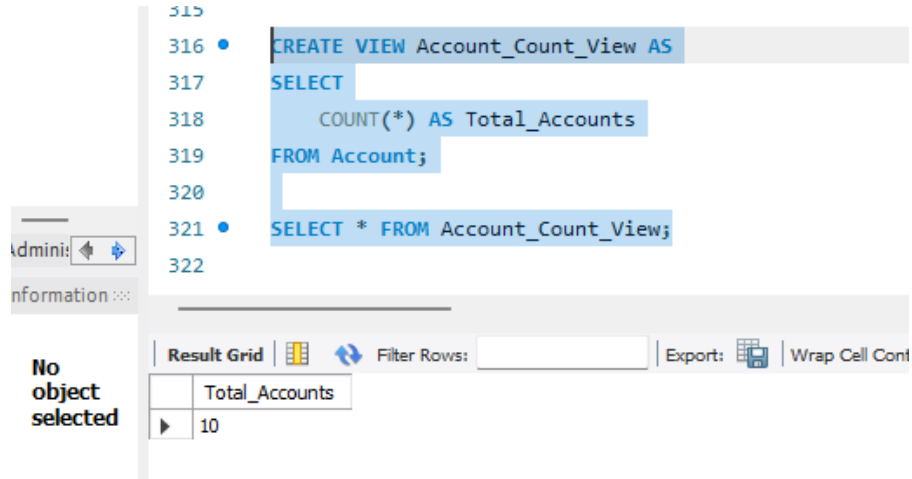
Minimum_Balance
35000.00

The interface also shows a toolbar with options like "Result Grid", "Filter Rows", "Export", and "Wrap Cell Content".

27. Total number of accounts :

```
CREATE VIEW Account_Count_View AS  
SELECT  
    COUNT(*) AS Total_Accounts  
FROM Account;
```

```
SELECT * FROM Account_Count_View;
```



28. Branch-wise account count :

```
CREATE VIEW Branch_Account_Count AS  
SELECT  
    Branch,  
    COUNT(*) AS Number_of_Accounts  
FROM Account  
GROUP BY Branch;
```

```
SELECT * FROM Branch_Account_Count;
```

```

322
323 • CREATE VIEW Branch_Account_Count AS
324 SELECT
325     Branch,
326     COUNT(*) AS Number_of_Accounts
327 FROM Account
328 GROUP BY Branch;
329
330 • SELECT * FROM Branch_Account_Count;
331

```

Admini:  

Information: 

No object selected

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

Branch	Number_of_Accounts
Hyderabad	3
Vijayawada	2
Bangalore	1
Chennai	1
Delhi	1
Mumbai	1
Pune	1

29. Branch wise total balance :

CREATE VIEW Branch_Total_Balance AS

SELECT

Branch,

SUM(Balance) AS Total_Balance

FROM Account

GROUP BY Branch;

SELECT * FROM Branch_Total_Balance;

```

332 • CREATE VIEW Branch_Total_Balance AS
333 SELECT
334     Branch,
335     SUM(Balance) AS Total_Balance
336 FROM Account
337 GROUP BY Branch;
338
339 • SELECT * FROM Branch_Total_Balance;
340

```

Admini:  

Information: 

No object selected

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

Branch	Total_Balance
Hyderabad	225000.00
Vijayawada	185000.00
Bangalore	120000.00
Chennai	45000.00
Delhi	65000.00
Mumbai	150000.00
Pune	35000.00

30. Account type count :

```
CREATE VIEW Account_Type_Count AS
SELECT
    Account_Type,
    COUNT(*) AS Number_of_Accounts
FROM Account
GROUP BY Account_Type;
```

```
SELECT * FROM Account_Type_Count;
```

The screenshot shows a SQL IDE interface. The main editor displays two SQL queries. The first query (lines 341-346) creates a view named 'Account_Type_Count' with columns 'Account_Type' and 'Number_of_Accounts'. The second query (line 348) selects all data from the 'Account_Type_Count' view. The status bar on the left indicates 'No object selected'. The bottom toolbar includes options for 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The 'Result Grid' is currently active, displaying the following data:

Account_Type	Number_of_Accounts
Savings	6
Current	4

31. Account type total balance :

```
CREATE VIEW Account_Type_Balance AS
SELECT
    Account_Type,
    SUM(Balance) AS Total_Balance
FROM Account
GROUP BY Account_Type;
```

```
SELECT * FROM Account_Type_Balance;
```

341 • CREATE VIEW Account_Type_Count AS
 342 SELECT
 343 Account_Type,
 344 COUNT(*) AS Number_of_Accounts
 345 FROM Account
 346 GROUP BY Account_Type;
 347
 348 • SELECT * FROM Account_Type_Count;
 349

Admin:  

Information 

No object selected

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

	Account_Type	Number_of_Accounts
▶	Savings	6
	Current	4

32. Branches with more than one account :

```
CREATE VIEW Multiple_Account_Branches AS
SELECT
    Branch,
    COUNT(*) AS Number_of_Accounts
FROM Account
GROUP BY Branch
HAVING COUNT(*) > 1;
SELECT * FROM Multiple_Account_Branches;
```

350 • CREATE VIEW Multiple_Account_Branches AS
 351 SELECT
 352 Branch,
 353 COUNT(*) AS Number_of_Accounts
 354 FROM Account
 355 GROUP BY Branch
 356 HAVING COUNT(*) > 1;
 357 • SELECT * FROM Multiple_Account_Branches;
 358
 359

dmini:  

Information 

No object selected

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

	Branch	Number_of_Accounts
▶	Hyderabad	3
	Vijayawada	2

33.Branches with total balance above 100000:

```
CREATE VIEW Rich_Branches AS
SELECT
    Branch,
    SUM(Balance) AS Total_Balance
FROM Account
GROUP BY Branch
HAVING SUM(Balance) > 100000;
```

```
SELECT * FROM Rich_Branches;
```

The screenshot shows a database management tool interface. The top pane displays SQL code for creating a view and querying it. The bottom pane shows the results of the query in a table format.

SQL Code:

```
358
359 • CREATE VIEW Rich_Branches AS
360 SELECT
361     Branch,
362     SUM(Balance) AS Total_Balance
363 FROM Account
364 GROUP BY Branch
365 HAVING SUM(Balance) > 100000;
366
367 • SELECT * FROM Rich_Branches;
368
```

Results:

Branch	Total_Balance
Hyderabad	225000.00
Vijayawada	185000.00
Bangalore	120000.00
Mumbai	150000.00

34. Deposit transaction view :

```
CREATE VIEW Deposit_Transaction_View AS
SELECT *
FROM Bank_Transaction
WHERE Transaction_Type = 'DEPOSIT';
```

```
SELECT * FROM Deposit_Transaction_View;
```



```

369 • CREATE VIEW Deposit_Transaction_View AS
370 SELECT *
371 FROM Bank_Transaction
372 WHERE Transaction_Type = 'DEPOSIT';
373
374 • SELECT * FROM Deposit_Transaction_View;
375

```

mini: [icon] [icon]
formation: [icon]

No object selected

Result Grid [icon] [icon] Filter Rows: [input] Export: [icon] Wrap Cell Content: [icon]

	Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
▶	1	10001	DEPOSIT	10000.00	2026-08-12 15:24:37
	3	10002	DEPOSIT	15000.00	2026-08-12 15:24:37
	5	10004	DEPOSIT	5000.00	2026-08-12 15:24:37
	7	10006	DEPOSIT	12000.00	2026-08-12 15:24:37
	8	10007	DEPOSIT	25000.00	2026-08-12 15:24:37
	10	10009	DEPOSIT	20000.00	2026-08-12 15:24:37

35. Withdrawal transaction view:

```

CREATE VIEW Withdrawal_Transaction_View AS
SELECT *
FROM Bank_Transaction
WHERE Transaction_Type = 'WITHDRAW';

```

```

SELECT * FROM Withdrawal_Transaction_View;

```

```

376 • CREATE VIEW Withdrawal_Transaction_View AS
377 SELECT *
378 FROM Bank_Transaction
379 WHERE Transaction_Type = 'WITHDRAW';
380
381 • SELECT * FROM Withdrawal_Transaction_View;
382

```

Admini: [icon] [icon]
Information: [icon]

No object selected

Result Grid [icon] [icon] Filter Rows: [input] Export: [icon] Wrap Cell Content: [icon]

	Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
▶	2	10001	WITHDRAW	5000.00	2026-08-12 15:24:37
	4	10003	WITHDRAW	20000.00	2026-08-12 15:24:37
	6	10005	WITHDRAW	10000.00	2026-08-12 15:24:37
	9	10008	WITHDRAW	5000.00	2026-08-12 15:24:37
	11	10010	WITHDRAW	15000.00	2026-08-12 15:24:37

36. Transaction details with customer:

```

CREATE VIEW Customer_Transaction_View AS
SELECT
    C.Customer_Name,
    A.Account_No,
    A.Account_Type,
    T.Transaction_ID,

```

```

T.Transaction_Type,
T.Amount,
T.Transaction_Date
FROM Customer C
JOIN Account A
ON C.Customer_ID = A.Customer_ID
JOIN Bank_Transaction T
ON A.Account_No = T.Account_No;

```

```
SELECT * FROM Customer_Transaction_View;
```

```

383 • CREATE VIEW Customer_Transaction_View AS
384 SELECT
385     C.Customer_Name,
386     A.Account_No,
387     A.Account_Type,
388     T.Transaction_ID,
389     T.Transaction_Type,
390     T.Amount,
391     T.Transaction_Date
392 FROM Customer C
393 JOIN Account A
394 ON C.Customer_ID = A.Customer_ID
395 JOIN Bank_Transaction T
396 ON A.Account_No = T.Account_No;
397
398 • SELECT * FROM Customer_Transaction_View;

```

	Customer_Name	Account_No	Account_Type	Transaction_ID	Transaction_Type	Amount	Transaction_Date
▶	Ravi Kumar	10001	Savings	1	DEPOSIT	10000.00	2026-08-12 15:24:
	Ravi Kumar	10001	Savings	2	WITHDRAW	5000.00	2026-08-12 15:24:
	Priya Sharma	10002	Savings	3	DEPOSIT	15000.00	2026-08-12 15:24:
	Arjun Reddy	10003	Current	4	WITHDRAW	20000.00	2026-08-12 15:24:
	Sneha Rao	10004	Savings	5	DEPOSIT	5000.00	2026-08-12 15:24:
	Kiran Kumar	10005	Current	6	WITHDRAW	10000.00	2026-08-12 15:24:
	Anil Kumar	10006	Savings	7	DEPOSIT	12000.00	2026-08-12 15:24:
	Meena Reddy	10007	Current	8	DEPOSIT	25000.00	2026-08-12 15:24:
	Rahul Sharma	10008	Savings	9	WITHDRAW	5000.00	2026-08-12 15:24:
	Lakshmi Devi	10009	Savings	10	DEPOSIT	20000.00	2026-08-12 15:24:
	Suresh Babu	10010	Current	11	WITHDRAW	15000.00	2026-08-12 15:24:

37. High value transaction view:

```

CREATE VIEW High_Value_Transaction_View AS
SELECT *
FROM Bank_Transaction
WHERE Amount > 10000;

```

```
SELECT * FROM High_Value_Transaction_View;
```

The screenshot shows a database management interface. On the left, a 'Schemas' pane is visible. The main area displays SQL code being executed. The code consists of two statements: a 'CREATE VIEW' statement and a 'SELECT' statement. The 'CREATE VIEW' statement creates a view named 'High_Value_Transaction_View' as a subset of 'Bank_Transaction' where the 'Amount' is greater than 10000. The 'SELECT' statement then queries this view. Below the code, a 'Result Grid' is shown, displaying the results of the 'SELECT' statement. The grid has columns for 'Transaction_ID', 'Account_No', 'Transaction_Type', 'Amount', and 'Transaction_Date'. It contains six rows of data. To the right of the grid, there are buttons for 'Export', 'Wrap Cell Content', 'Form Editor', and a 'Result Grid' button.

```
400 • CREATE VIEW High_Value_Transaction_View AS
401 SELECT *
402 FROM Bank_Transaction
403 WHERE Amount > 10000;
404
405 • SELECT * FROM High_Value_Transaction_View;
406
407
```

ct selected

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
3	10002	DEPOSIT	15000.00	2026-08-12 15:24:37
4	10003	WITHDRAW	20000.00	2026-08-12 15:24:37
7	10006	DEPOSIT	12000.00	2026-08-12 15:24:37
8	10007	DEPOSIT	25000.00	2026-08-12 15:24:37
10	10009	DEPOSIT	20000.00	2026-08-12 15:24:37
11	10010	WITHDRAW	15000.00	2026-08-12 15:24:37

38. Accounts ordered by balance :

```
CREATE VIEW Balance_Ranking_View AS
```

```
SELECT
```

```
    Account_No,
```

```
    Customer_ID,
```

```
    Account_Type,
```

```
    Balance
```

```
FROM Account
```

```
ORDER BY Balance DESC;
```

```
SELECT * FROM Balance_Ranking_View;
```

kludb

sys

Administration

Schemas

Information

No object selected

```

406
407 • CREATE VIEW Balance_Ranking_View AS
408 SELECT
409     Account_No,
410     Customer_ID,
411     Account_Type,
412     Balance
413 FROM Account
414 ORDER BY Balance DESC;
415
416 • SELECT * FROM Balance_Ranking_View;
417

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Account_No	Customer_ID	Account_Type	Balance
▶	10007	107	Current	150000.00
	10003	103	Current	120000.00
	10010	110	Current	110000.00
	10005	105	Current	90000.00
	10009	109	Savings	85000.00
	10002	102	Savings	75000.00
	10006	106	Savings	65000.00
	10001	101	Savings	50000.00
	10004	104	Savings	45000.00
	10008	108	Savings	35000.00

39. Customer orders by names:

```
CREATE VIEW Customer_Name_View AS
```

```
SELECT
```

```
    Customer_ID,
```

```
    Customer_Name,
```

```
    City
```

```
FROM Customer
```

```
ORDER BY Customer_Name;
```

```
SELECT * FROM Customer_Name_View;
```

418 • CREATE VIEW Customer_Name_View AS
 419 SELECT
 420 Customer_ID,
 421 Customer_Name,
 422 City
 423 FROM Customer
 424 ORDER BY Customer_Name;
 425
 426 • SELECT * FROM Customer_Name_View;
 427

tion Schemas

ject selected

Result Grid Filter Rows: Export: Wrap Cell C

	Customer_ID	Customer_Name	City
▶	106	Anil Kumar	Delhi
	103	Arjun Reddy	Bangalore
	105	Kiran Kumar	Hyderabad
	109	Lakshmi Devi	Hyderabad
	107	Meena Reddy	Mumbai
	102	Priya Sharma	Vijayawada
	108	Rahul Sharma	Pune
	101	Ravi Kumar	Hyderabad
	104	Sneha Rao	Chennai
	110	Suresh Babu	Vijayawada

40. Loan interest calculation view:

```
CREATE VIEW Loan_Interest_View AS
SELECT
```

```
    Loan_ID,
    Customer_ID,
    Loan_Type,
    Loan_Amount,
    Interest_Rate,
    (Loan_Amount * Interest_Rate / 100)
    AS Annual_Interest
```

```
FROM Loan;
```

```
SELECT * FROM Loan_Interest_View;
```

```

428 • CREATE VIEW Loan_Interest_View AS
429 SELECT
430     Loan_ID,
431     Customer_ID,
432     Loan_Type,
433     Loan_Amount,
434     Interest_Rate,
435     (Loan_Amount * Interest_Rate / 100)
436     AS Annual_Interest
437 FROM Loan;
438
439 • SELECT * FROM Loan_Interest_View;
440

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate	Annual_Interest
▶	501	101	Home Loan	5000000.00	7.50	375000.00000
	502	102	Education Loan	1000000.00	6.50	65000.00000
	503	103	Car Loan	800000.00	8.20	65600.00000
	504	104	Personal Loan	500000.00	10.50	52500.00000
	505	105	Home Loan	4000000.00	7.20	288000.00000
	506	106	Car Loan	900000.00	8.50	76500.00000
	507	107	Business Loan	3000000.00	9.00	270000.00000
	508	109	Personal Loan	600000.00	10.00	60000.00000

Result Grid
Form Editor
Field Types

41. Loan total amount view:

```

CREATE VIEW Loan_Total_Amount_View AS
SELECT

```

```

    Loan_ID,
    Customer_ID,
    Loan_Type,
    Loan_Amount,
    Interest_Rate,
    Loan_Amount +
    (Loan_Amount * Interest_Rate / 100)
    AS Total_Amount

```

```

FROM Loan;

```

```

SELECT * FROM Loan_Total_Amount_View;

```

```

440 • CREATE VIEW Loan_Total_Amount_View AS
441 SELECT
442     Loan_ID,
443     Customer_ID,
444     Loan_Type,
445     Loan_Amount,
446     Interest_Rate,
447     Loan_Amount +
448     (Loan_Amount * Interest_Rate / 100)
449     AS Total_Amount
450 FROM Loan;
451
452 • SELECT * FROM Loan_Total_Amount_View;
453

```

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate	Total_Amount
501	101	Home Loan	5000000.00	7.50	5375000.0000
502	102	Education Loan	1000000.00	6.50	1065000.0000
503	103	Car Loan	800000.00	8.20	865600.0000
504	104	Personal Loan	500000.00	10.50	552500.0000
505	105	Home Loan	4000000.00	7.20	4288000.0000
506	106	Car Loan	900000.00	8.50	976500.0000
507	107	Business Loan	3000000.00	9.00	3270000.0000
508	109	Personal Loan	600000.00	10.00	660000.0000

42. Display high balance accounts from view:

```

SELECT *
FROM High_Balance_View
WHERE Balance > 120000;

```

```

454 • SELECT *
455 FROM High_Balance_View
456 WHERE Balance > 120000;
457

```

Account_No	Customer_ID	Account_Type	Balance
10007	107	Current	150000.00

43. Display savings account above 60000:

```

SELECT *
FROM Savings_Account_View
WHERE Balance > 60000;

```

```

458 • SELECT *
459 FROM Savings_Account_View
460 WHERE Balance > 60000;
461

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10002	102	Savings	75000.00	Vijayawada
	10006	106	Savings	65000.00	Delhi
	10009	109	Savings	85000.00	Hyderabad

Result Grid
Form Editor

44. Display hyderabad accounts above 50000:

```

SELECT *
FROM Hyderabad_Account_View
WHERE Balance > 50000;

```

```

462 • SELECT *
463 FROM Hyderabad_Account_View
464 WHERE Balance > 50000;
465

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10005	105	Current	90000.00	Hyderabad
	10009	109	Savings	85000.00	Hyderabad

Result Grid
Form Editor

45. Display customers from hyderabad:

```

SELECT *
FROM Customer_Basic_View
WHERE City = 'Hyderabad';

```

```

466 • SELECT *
467 FROM Customer_Basic_View
468 WHERE City = 'Hyderabad';
469

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Customer_ID	Customer_Name	City
▶	101	Ravi Kumar	Hyderabad
	105	Kiran Kumar	Hyderabad
	109	Lakshmi Devi	Hyderabad

46. Display customers accounts ordered by balance:

```
SELECT *  
FROM Customer_Account_View  
ORDER BY Balance DESC;
```

470 •

```
SELECT *
```

471

```
FROM Customer_Account_View
```

472

```
ORDER BY Balance DESC;
```

473

474

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch
▶	107	Meena Reddy	Mumbai	10007	Current	150000.00	Mumbai
	103	Arjun Reddy	Bangalore	10003	Current	120000.00	Bangalore
	110	Suresh Babu	Vijayawada	10010	Current	110000.00	Vijayawada
	105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad
	109	Lakshmi Devi	Hyderabad	10009	Savings	85000.00	Hyderabad
	102	Priya Sharma	Vijayawada	10002	Savings	75000.00	Vijayawada
	106	Anil Kumar	Delhi	10006	Savings	65000.00	Delhi
	101	Ravi Kumar	Hyderabad	10001	Savings	50000.00	Hyderabad
	104	Sneha Rao	Chennai	10004	Savings	45000.00	Chennai
	108	Rahul Sharma	Pune	10008	Savings	35000.00	Pune

47. Display loans above 1000000:

```
SELECT *  
FROM Customer_Loan_View  
WHERE Loan_Amount > 1000000;
```

```
474 • SELECT *
475 FROM Customer_Loan_View
476 WHERE Loan_Amount > 1000000;
477
```

Customer_ID	Customer_Name	City	Loan_ID	Loan_Type	Loan_Amount	Interest
101	Ravi Kumar	Hyderabad	501	Home Loan	5000000.00	7.50
105	Kiran Kumar	Hyderabad	505	Home Loan	4000000.00	7.20
107	Meena Reddy	Mumbai	507	Business Loan	3000000.00	9.00

48. Update balance through view:

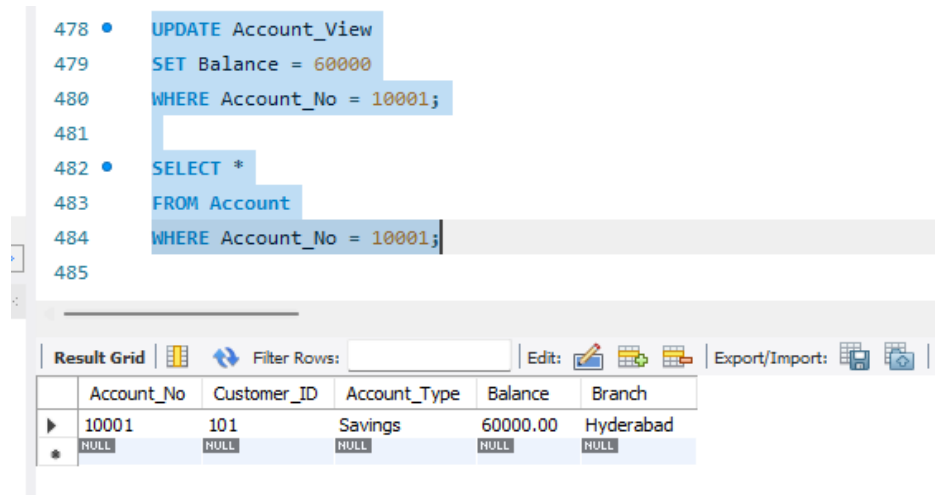
```
UPDATE Account_View  
SET Balance = 60000
```

```
WHERE Account_No = 10001;
```

```
SELECT *
```

```
FROM Account
```

```
WHERE Account_No = 10001;
```



49. Insert using simple view:

```
CREATE VIEW Simple_Account_View AS
```

```
SELECT
```

```
    Account_No,
```

```
    Customer_ID,
```

```
    Account_Type,
```

```
    Balance,
```

```
    Branch
```

```
FROM Account;
```

```
INSERT INTO Simple_Account_View
```

```
VALUES
```

```
(10011, 101, 'Savings', 55000, 'Hyderabad');
```

```
SELECT *
```

```
FROM Account;
```

```

486 • CREATE VIEW Simple_Account_View AS
487 SELECT
488     Account_No,
489     Customer_ID,
490     Account_Type,
491     Balance,
492     Branch
493 FROM Account;
494
495 • INSERT INTO Simple_Account_View
496 VALUES
497 (10011, 101, 'Savings', 55000, 'Hyderabad');
498
499 • SELECT *
500 FROM Account;
501

```

Result Grid					
Filter Rows:					
Edit:					
Export/Import:					
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	60000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
	10003	103	Current	120000.00	Bangalore
	10004	104	Savings	45000.00	Chennai
	10005	105	Current	90000.00	Hyderabad
	10006	106	Savings	65000.00	Delhi
	10007	107	Current	150000.00	Mumbai
	10008	108	Savings	35000.00	Pune
	10009	109	Savings	85000.00	Hyderabad
	10010	110	Current	110000.00	Vijayawada
	10011	101	Savings	55000.00	Hyderabad
*	NULL	NULL	NULL	NULL	NULL

50. Delete using simple view:

```

DELETE FROM Simple_Account_View
WHERE Account_No = 10011;

```

```

SELECT *
FROM Account;

```

```

502 • DELETE FROM Simple_Account_View
503 WHERE Account_No = 10011;
504
505 • SELECT *
506 FROM Account;
507
508

```

Result Grid

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	60000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
	10003	103	Current	120000.00	Bangalore
	10004	104	Savings	45000.00	Chennai
	10005	105	Current	90000.00	Hyderabad
	10006	106	Savings	65000.00	Delhi
	10007	107	Current	150000.00	Mumbai
	10008	108	Savings	35000.00	Pune
	10009	109	Savings	85000.00	Hyderabad
	10010	110	Current	110000.00	Vijayawada
*	NULL	NULL	NULL	NULL	NULL

51. Show all views:

SHOW FULL TABLES

WHERE TABLE_TYPE = 'VIEW';

```

506 FROM Account;
507
508 • SHOW FULL TABLES
509 WHERE TABLE_TYPE = 'VIEW';
510

```

administrativ
information

No object selected

Result Grid

Tables_in_bankdb	Table_type
▶ account_count_view	VIEW
account_type_count	VIEW
account_view	VIEW
average_account_balance	VIEW
balance_ranking_view	VIEW
branch_account_count	VIEW
branch_total_balance	VIEW
current_account_view	VIEW
customer_account_view	VIEW
customer_banking_view	VIEW
customer_basic_view	VIEW
customer_loan_view	VIEW
customer_name_view	VIEW
customer_transaction_view	VIEW
customer_view	VIEW
deposit_transaction_view	VIEW

Result 28 x

52. Show view definition:

SHOW CREATE VIEW Customer_Account_View;

511 • `SHOW CREATE VIEW Customer_Account_View;`

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
View	Create View	character_set_client	col
customer_account_view	CREATE ALGORITHM=UNDEFINED DEFINER=`r...`	utf8mb4	utf8

53. Describe view:

DESCRIBE Customer_Account_View;

512 • `DESCRIBE Customer_Account_View;`

Result Grid	Filter Rows:	Export:	Wrap Cell Content:		
Field	Type	Null	Key	Default	Extra
Customer_ID	int	NO		NULL	
Customer_Name	varchar(100)	NO		NULL	
City	varchar(50)	YES		NULL	
Account_No	int	NO		NULL	
Account_Type	varchar(20)	YES		NULL	
Balance	decimal(12,2)	YES		NULL	
Branch	varchar(50)	YES		NULL	

54. Drop one view:

DROP VIEW Customer_View;

514 • DROP VIEW Customer_View;

Output

Action Output

#	Time	Action	Message
✓ 46	20:34:09	CREATE VIEW Simple_Account_View AS SELECT Acc...	0 row(s) affected
✓ 47	20:34:10	INSERT INTO Simple_Account_View VALUES (10011, 10...	1 row(s) affected
✓ 48	20:34:10	SELECT * FROM Account LIMIT 0, 1000	11 row(s) returned
✓ 49	20:34:35	DELETE FROM Simple_Account_View WHERE Account_...	1 row(s) affected
✓ 50	20:34:35	SELECT * FROM Account LIMIT 0, 1000	10 row(s) returned
✓ 51	20:34:56	SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW'	31 row(s) returned
✓ 52	20:35:30	SHOW CREATE VIEW Customer_Account_View	1 row(s) returned
✓ 53	20:35:53	DESCRIBE Customer_Account_View	7 row(s) returned
✓ 54	20:36:13	DROP VIEW Customer_View	0 row(s) affected

55. Drop multiple views:

DROP VIEW

Customer_Basic_View,

Savings_Account_View,

Current_Account_View;

```
515 • DROP VIEW
516 Customer_Basic_View,
517 Savings_Account_View,
518 Current_Account_View;
519
```

Output



Action Output

	#	Time	Action	Message
✓	47	20:34:10	INSERT INTO Simple_Account_View VALUES (10011, 10...	1 row(s) affected
✓	48	20:34:10	SELECT * FROM Account LIMIT 0, 1000	11 row(s) returned
✓	49	20:34:35	DELETE FROM Simple_Account_View WHERE Account_...	1 row(s) affected
✓	50	20:34:35	SELECT * FROM Account LIMIT 0, 1000	10 row(s) returned
✓	51	20:34:56	SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW'	31 row(s) returned
✓	52	20:35:30	SHOW CREATE VIEW Customer_Account_View	1 row(s) returned
✓	53	20:35:53	DESCRIBE Customer_Account_View	7 row(s) returned
✓	54	20:36:13	DROP VIEW Customer_View	0 row(s) affected
✓	55	20:36:48	DROP VIEW Customer_Basic_View, Savings_Account_Vie...	0 row(s) affected